VecinApp

https://github.com/TejonSereno/ProyectoFinal

Nome Alumno/a: ***Brais Fernandez Vazquez***

Curso: ***2º DAM***

Módulo: ***Proxecto Final Ciclo***

Contido

Contido

1. Introducion 3
2. Obxectivos 3
3. Situacion Previa 4
4. Tecnoloxías empregadas 4
5. Solucion Proposta 6
6. Presuposto 7
7. Planificacion do proyecto 10

1. Introducción

O proxecto desenvolvido consiste nunha aplicación para a xestión de comunidades de veciños, destinada a centralizar nun único sistema diferentes tarefas relacionadas coa administración e comunicación dunha comunidade.

A aplicación está composta por unha aplicación móbil desenvolvida con Flutter e Dart, que actúa como interface para os usuarios, e un backend desenvolvido en Java mediante Spring Boot, encargado de xestionar a lóxica da aplicación, a seguridade e o acceso aos datos. A información almacénase nunha base de datos PostgreSQL.

O sistema permite diferenciar entre distintos tipos de usuarios, como administradores e veciños, ofrecendo a cada perfil as funcionalidades correspondentes.

2. Obxetivos

- Desenvolver unha aplicación para facilitar a xestión dunha comunidade de veciños.
- Crear unha aplicación móbil accesible desde dispositivos Android.
- Desenvolver un backend que centralice a lóxica da aplicación.
- Separar correctamente a aplicación en diferentes capas.
- Implementar un sistema de autenticación e autorización de usuarios.
- Diferenciar as funcionalidades dispoñibles segundo o tipo de usuario.
- Despregar a aplicación nun contorno accesible desde Internet.

3. Situación previa

Nunha comunidade que non dispón dunha ferramenta específica, a información pode estar repartida entre diferentes medios: grupos de mensaxería, correos electrónicos, documentos impresos ou follas de cálculo. Isto pode provocar que a información quede dispersa e que resulte máis complicado consultala.

O problema que se pretende abordar con este proxecto é a necesidade de dispoñer dun sistema centralizado que permita xestionar unha comunidade de veciños desde unha aplicación informática.

Existen diferentes alternativas para resolver este problema.

Alternativas existentes

Unha primeira alternativa consiste en empregar ferramentas xenéricas como follas de cálculo, correo electrónico ou calendarios compartidos. Estas ferramentas teñen a vantaxe de ser coñecidas, pero non están deseñadas especificamente para a xestión dunha comunidade.

Outra opción son as aplicacións especializadas en comunidades de propietarios. Actualmente existen solucións como **TusVecinos**, **Fincapp**, **Plusvecinos** ou **Comunya**, que ofrecen diferentes combinacións de comunicación, documentación, incidencias, reservas e outras funcionalidades.

Tamén existen solucións centradas especificamente na administración profesional de fincas, que engaden funcionalidades económicas e administrativas máis avanzadas.

4. Tecnoloxías empregadas

Backend

Java 17

Java é a linguaxe empregada para desenvolver o backend da aplicación. A versión utilizada é Java 17.

Spring Boot 3

Spring Boot emprégase como framework principal para desenvolver a API REST do servidor. Permite estruturar a aplicación e simplificar a configuración dos diferentes compoñentes.

Spring Security

Spring Security utilízase para implementar os mecanismos de seguridade e control de acceso da aplicación.

JWT

JSON Web Token emprégase para realizar a autenticación.

Maven

Maven utilízase para a xestión do proxecto Java e das súas dependencias.

Frontend

Flutter

Flutter é o framework empregado para desenvolver a aplicación móbil. Permite crear a interface da aplicación a partir de widgets e facilita o desenvolvemento multiplataforma.

Dart

Dart é a linguaxe de programación utilizada por Flutter.

Base de datos

PostgreSQL

PostgreSQL é o sistema de xestión de bases de datos relacionais utilizado polo proxecto. A base de datos almacena a información necesaria para o funcionamento da aplicación.

DevOps e ferramentas auxiliares

Docker

Docker utilízase para empaquetar e executar o backend nun contedor, facilitando o seu despregamento.

Git e GitHub

Git utilízase como sistema de control de versións e GitHub como plataforma para almacenar o código fonte e facilitar a xestión do proxecto.

Render

Render utilízase para realizar o despregamento do backend. O repositorio indica que o servizo se constrúe automaticamente a partir de GitHub e que se utilizan variables de ambiente para configurar a conexión coa base de datos.

5. Solución proposta

A solución proposta consiste nunha arquitectura **cliente-servidor**, formada por tres elementos principais:

1. Aplicación móbil desenvolvida con Flutter.
2. API REST desenvolvida con Spring Boot.
3. Base de datos PostgreSQL.

O fluxo de funcionamento é o seguinte:

Aplicación Flutter → API Spring Boot → PostgreSQL

Cando o usuario realiza unha operación desde a aplicación móbil, a petición envíase ao backend. O backend comproba a autenticación e autorización do usuario, executa a lóxica de negocio correspondente e accede á base de datos cando é necesario.

A aplicación emprega unha arquitectura por capas no backend:

Controller → Service → Repository → Database

Esta separación permite que cada capa teña unha responsabilidade concreta. Os controladores reciben as peticións, os servizos conteñen a lóxica de negocio, os repositorios realizan o acceso aos datos e PostgreSQL encárgase do almacenamento persistente.

Unha das características da solución é que non se trata simplemente dunha aplicación local. O backend está despregado nun servidor e a base de datos nun servizo externo, polo que a aplicación móbil pode comunicarse coa infraestrutura a través de Internet.

A solución tamén está preparada para facilitar o desenvolvemento e mantemento posterior, xa que a API está documentada mediante Swagger/OpenAPI e o backend está empaquetado mediante Docker.

6. Presiposto

O orzamento pode dividirse en tres categorías: hardware, software/infraestrutura e recursos humanos.

Hardware

Para o desenvolvemento do proxecto é necesario dispoñer dun ordenador capaz de executar o entorno de desenvolvemento

Como estimación:

Recurso	***Custo estimado***
Portatil de desarrolo	*900€*
Dispositivo de pruevas	*300€*
Total hardware	***1200€***

Software e infraestrutura

As principais ferramentas utilizadas dispoñen de versións gratuítas ou son software libre. GitHub ofrece GitHub Free para repositorios públicos e colaboración

Servizo	***Custo mensual***	***Custo anual***
Render(Plan pro)	*25€*	*300€*
Suprabase(PLan pro)	*25€*	*300€*
Total Sofware	***50€***	*600€*

* O custo real pode variar segundo o consumo e o plan contratado.

Render indica que os servizos gratuítos están pensados principalmente para probas, proxectos persoais e desenvolvemento, e presentan limitacións como a suspensión por inactividade. Isto coincide coa limitación indicada no propio README do proxecto.

Recursos humanos

Para calcular o custo dos recursos humanos pódese realizar unha estimación baseada nas horas dedicadas ao desenvolvemento.

Por exemplo, considerando aproximadamente **150 horas de traballo** e un custo orientativo de **25 €/hora**:

150 horas × 25 €/hora = 3.750 €

Resumo do orzamento

Recurso	***Custo estimado***
Harware	*1200€*
Sofware	*600€*
Recursos humanos	*3.750€*
Total	***5.550€***

7. Planificación do proxecto

Diagrama de Gantt

O diagrama de Gantt representa graficamente a planificación temporal do proxecto nun plazo de 12 semanas. Nel móstranse as diferentes tarefas realizadas durante o desenvolvemento e o período de tempo estimado ou empregado para cada unha delas.

Tarefa	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11	S12
Análise do problema	■	■										
Definición de requisitos	■	■	■									
Deseño da arquitectura		■	■									
Deseño da base de datos			■	■								
Desenvolvemento do backend				■	■	■	■					
Desenvolvemento da aplicación Flutter					■	■	■	■				
Autenticación e seguridade						■	■					
Integración frontend-backend							■	■				
Docker e despregamento								■	■			
Probas									■	■		
Corrección de erros										■	■	
Documentación										■	■	■
Preparación da versión final												■

Descripción do modelo de desenvolvemento de software a implementar

Para o desenvolvemento do proxecto emprégase un modelo iterativo e incremental.

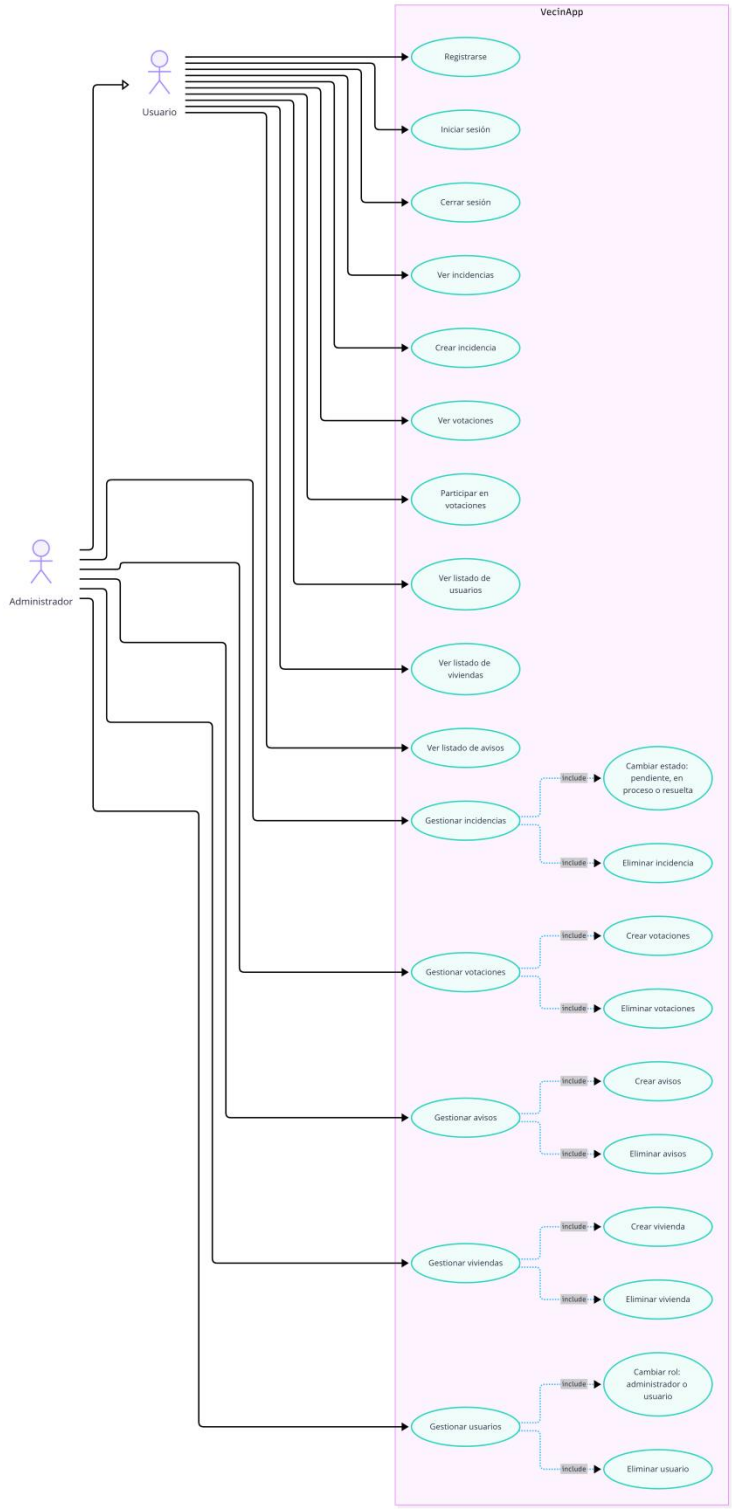
Este modelo consiste en dividir o desenvolvemento en diferentes fases ou iteracións, permitindo implementar progresivamente as funcionalidades da aplicación. Desta maneira, é posible probar cada parte do sistema antes de continuar co desenvolvemento das seguintes funcionalidades.

Inicialmente realízase a análise do problema e a definición dos requisitos. Posteriormente deseñanse a arquitectura, a base de datos e a interface da aplicación.

A continuación desenvolvese o backend e o frontend de maneira progresiva, realizando posteriormente a integración entre ambos. Finalmente realízanse as probas, a corrección de erros, o despregamento e a documentación.

Diagrama de casos de usos

O diagrama de casos de uso representa as interaccións entre os diferentes tipos de usuarios da aplicación e as funcionalidades que ofrece o sistema.



Universo de discurso da base de datos.

O sistema permite xestionar diferentes **comunidades**, as **vivendas** que pertencen a cada comunidade e os **usuarios** asociados a elas. Ademais, os usuarios poden participar en **votacións**, rexistrando o seu voto entre as diferentes opcións dispoñibles.

As comunidades tamén poden publicar **avisos** para informar aos seus membros sobre diferentes asuntos e xestionar **incidencias** relacionadas coa comunidade

Elementos principais

Os **usuarios** representan as persoas que utilizan o sistema. Cada usuario dispón dun identificador único e almacena información como o nome, correo electrónico, contrasinal e rol dentro do sistema.

As **comunidades** representan as comunidades de propietarios xestionadas polo sistema. Cada comunidade ten un identificador único e un nome.

As **vivendas** representan os inmobles pertencentes ás comunidades. Cada vivenda ten un identificador, un número, unha rúa e unha descrición.

As **incidencias** permiten rexistrar problemas ou situacións que afectan a unha comunidade. Cada incidencia dispón dun identificador, título, descrición, estado, data de creación e data de finalización.

Os **avisos** son comunicacións realizadas dentro dunha comunidade para informar aos usuarios sobre asuntos relevantes.

As **votacións** permiten aos membros dunha comunidade tomar decisións mediante un proceso de votación.

A entidade **Voto** representa a participación dun usuario nunha votación. Cada voto identifica o usuario que realiza a votación e a votación na que participa, ademais da opción seleccionada.

Votación_opción almacena as diferentes opcións dispoñibles para cada votación. Cada opción está identificada mediante un identificador de votación e contén o texto ou valor da opción.

Relacións entre elementos

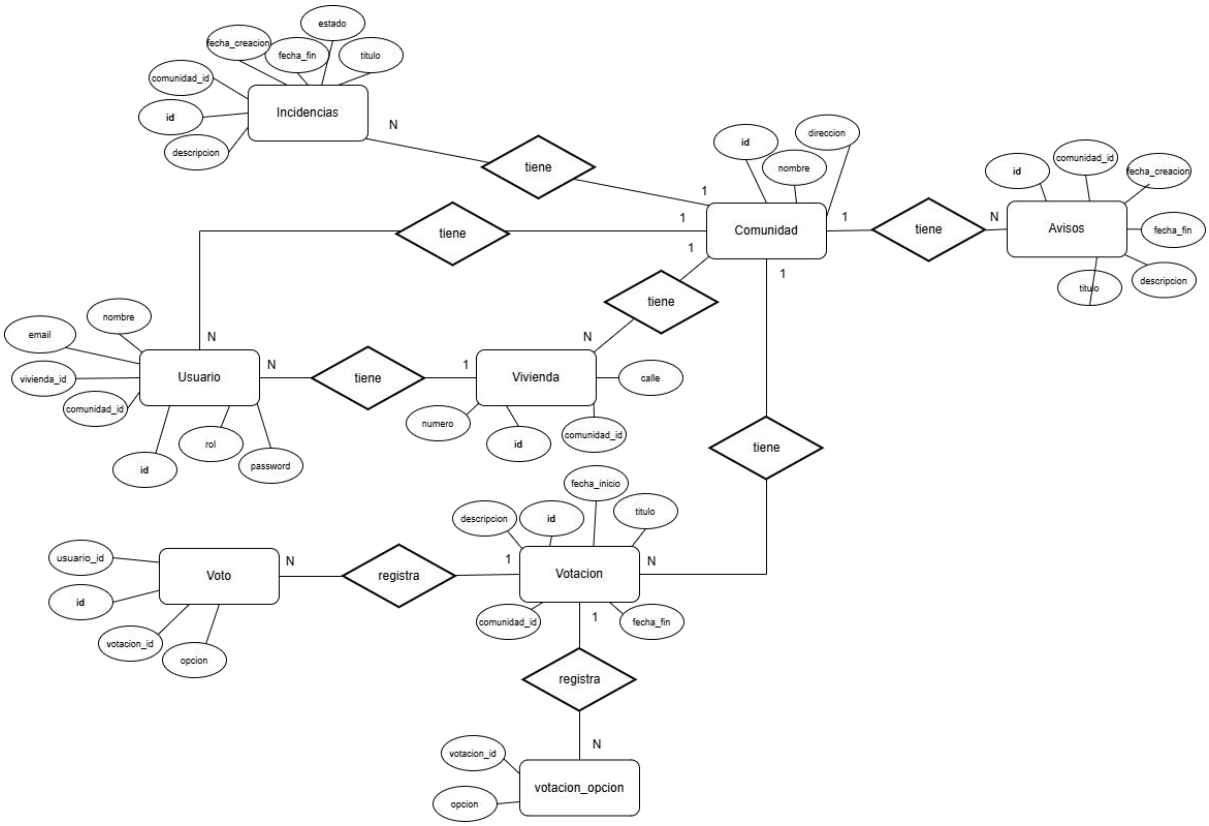
- Unha **comunidade pode ter varias vivendas**, pero cada vivenda pertence a unha única comunidade.
- Unha **comunidade pode ter varios usuarios**, e os usuarios están asociados a unha comunidade.
- Unha **comunidade pode ter varias incidencias**, pero cada incidencia pertence a unha comunidade.
- Unha **comunidade pode ter varios avisos**, pero cada aviso está asociado a unha comunidade.
- Unha **comunidade pode ter varias votacións**, pero cada votación pertence a unha comunidade.
- Unha **votación pode ter varias opcións**, almacenadas na entidade VOTACION_OPCION.
- Un **usuario pode rexistrar votos en diferentes votacións**.
- Cada **voto está asociado a un usuario e a unha votación**.
- Cada voto selecciona unha **opción pertencente á votación correspondente**.

Regras de xestión

- Unha vivenda non pode pertencer a máis dunha comunidade.
- Cada incidencia debe estar asociada a unha comunidade.
- Cada aviso debe estar asociado a unha comunidade.
- Unha votación debe pertencer a unha comunidade.
- As votacións só deben aceptar votos durante o período comprendido entre a súa data de inicio e a súa data de fin.
- Un usuario non debería poder rexistrar máis dun voto para a mesma votación.
- Un voto debe corresponder a unha opción existente da votación na que se realiza.

- As opcións dunha votación deben estar asociadas á súa votación correspondente.
- Os usuarios autenticados poderán realizar as accións permitidas segundo o seu rol.

Diagrama de Entidad-Relación.



Modelo relacional da base de datos.

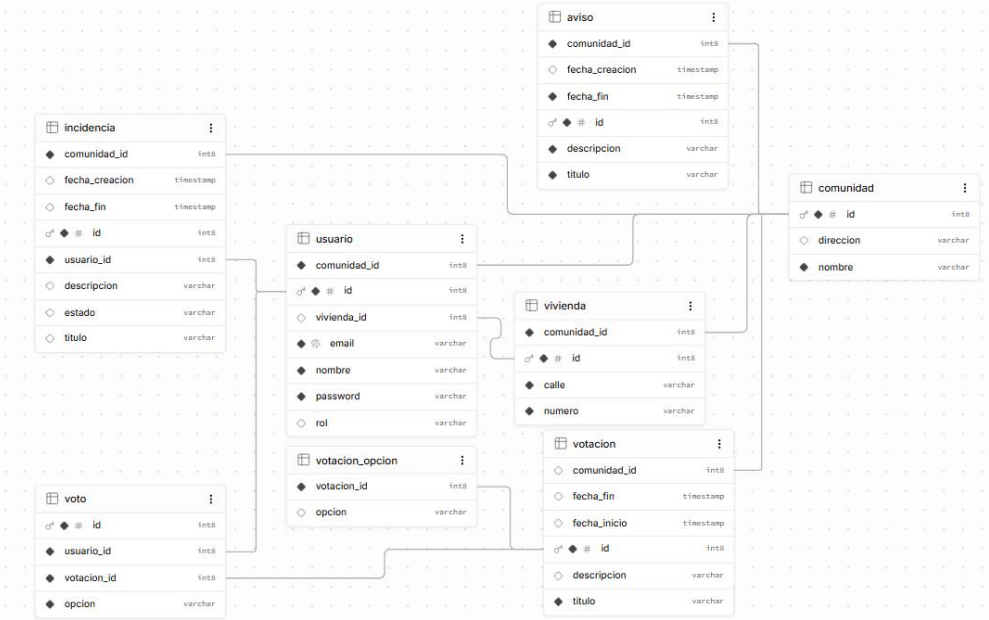
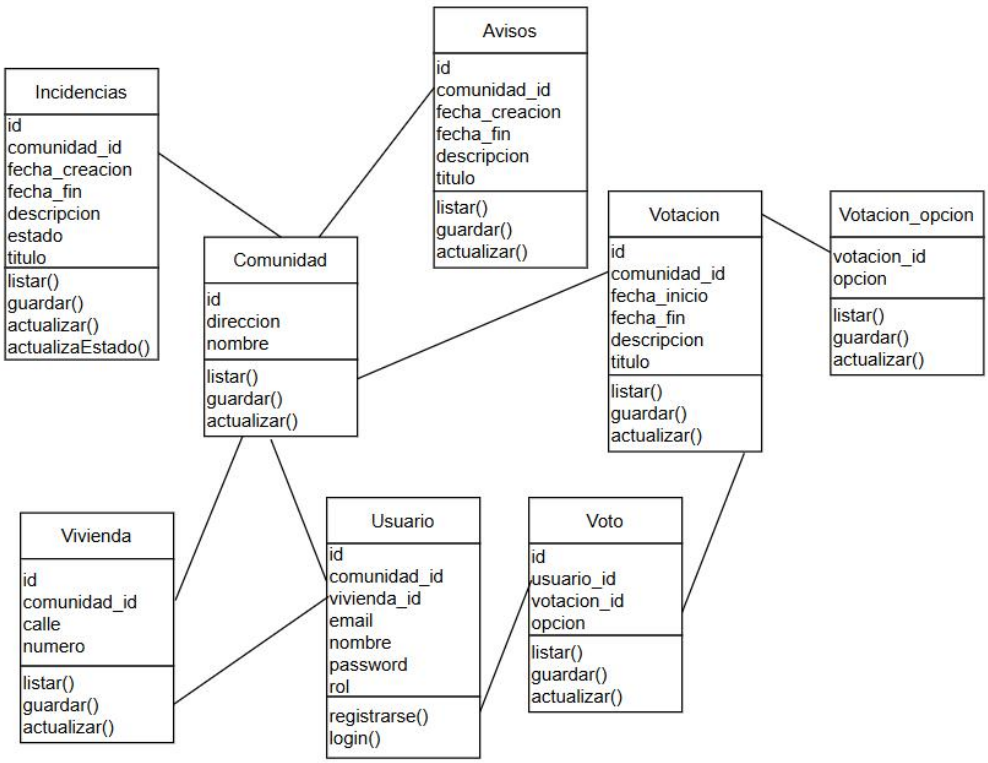
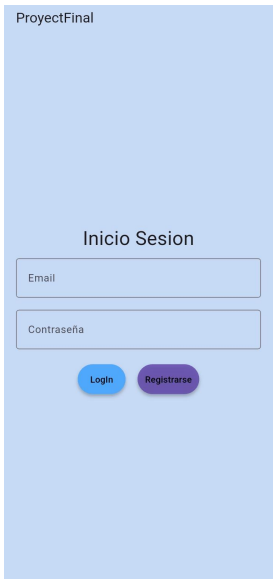


Diagrama de clases.

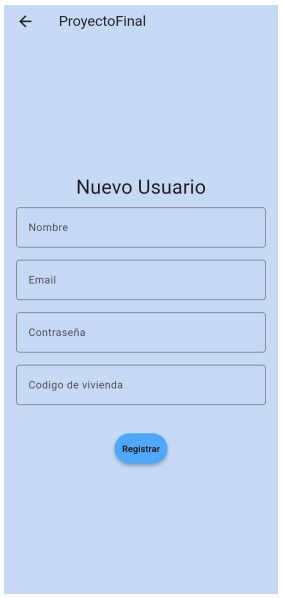


Mockups da interface

Inicio de sesio



Pantalla de registro



Pantalla principal

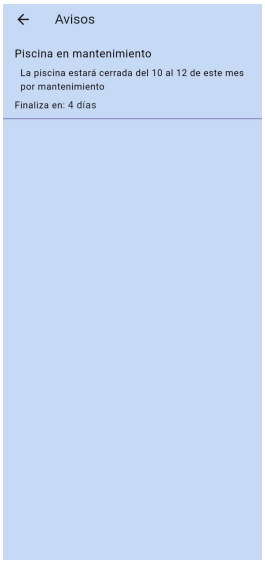


Pantalla perfil

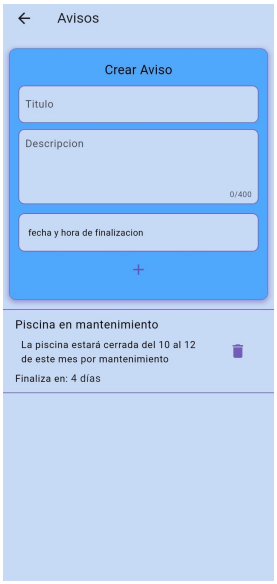


Pantalla Avisos

Usuario

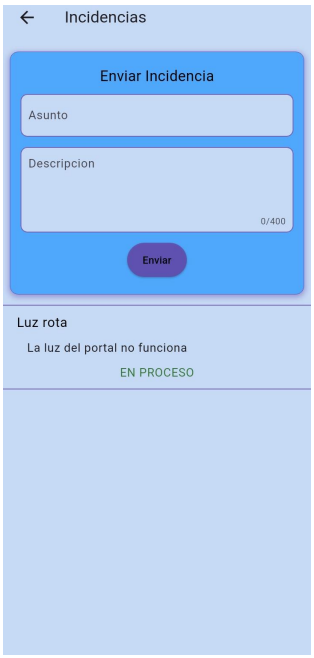


Administrador



Pantalla Incidencias

Usuario

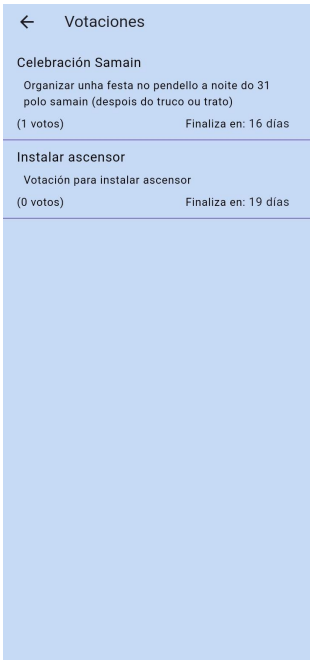


Administrador

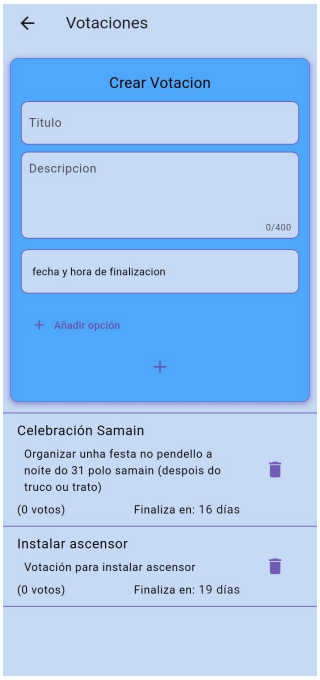


Pantalla Votaciones

Usuario

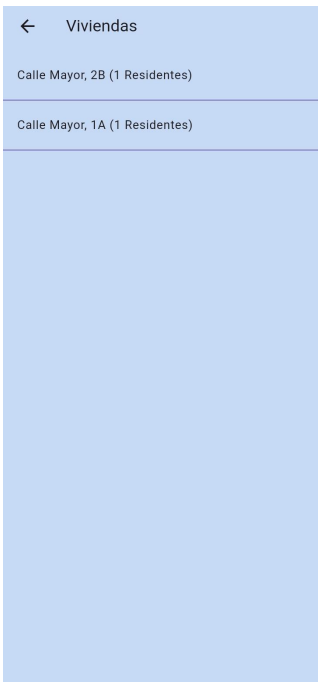


Administrador

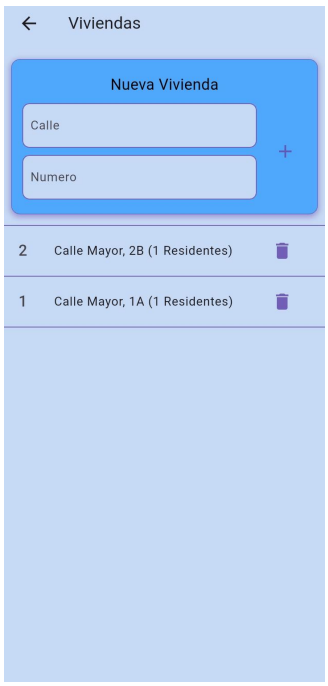


Pantalla Viviendas

Usuario

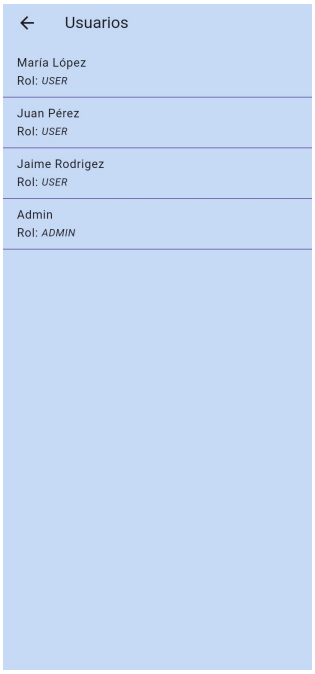


Adminstrado



Pantalla Usuarios

Usuarios



Administrador

